

AMCL 파라미터 & 글로벌 로컬라이제이션

Adaptive Monte Carlo Localization

cchyun@gmail.com

Part 1. AMCL 핵심 파라미터 튜닝

AMCL이란 무엇인가?

- AMCL(Adaptive Monte Carlo Localization) — 이미 알고 있는 지도에서 내 위치를 찾는 알고리즘
 - 비유: 눈을 가리고 집 안에서 벽을 더듬으며 위치 파악
 - "아, 여기가 거실 벽이구나" → 센서 데이터 + 지도 비교로 내 위치 추정
 - 파티클(Particle) 기반 추정
 - 수많은 가상의 위치 후보(파티클)를 지도 위에 뿌림
 - 로봇의 센서 데이터와 지도를 비교하며 가장 확률이 높은 곳으로 파티클을 수렴
 - 출력: 로봇의 현재 위치 (x, y, θ)

Part 1-1. 파티클 관련 파라미터

- 파티클(Particle): 로봇의 위치 후보 — 얼마나 많이, 어떻게 관리할지 결정
 - `min_particles` / `max_particles` (최소 / 최대 파티클 수)
 - 보통 500 ~ 2000개 사이로 설정
 - 트레이드오프

파티클 수	정확도	CPU 연산량
많음	↑ 높음	↑ 급증
적음	↓ 낮음	↓ 감소

- 위치를 확신할 때 → `min` 유지 (효율) - 위치를 잃어버렸을 때 → `max` 사용 (넓은 범위 탐색)

- `pf_err` (`kld_err`) / `pf_z` (`kld_z`) — KLD-Sampling 수렴 조건
 - "Adaptive(적응형)"이라는 이름이 붙은 이유
 - 실제 확률 분포와 파티클로 근사한 분포 사이의 오차(KLD)를 계산
 - KLD가 `pf_err` 이하이고, 신뢰도가 `pf_z` 이상이면 파티클 생성을 스스로 멈춤
 - 효과
 - 로봇이 위치를 잘 잡았다 → 파티클 수 자동 감소 → **효율 증가**
 - 위치가 불확실하다 → 파티클 수 자동 증가 → **정확도 확보**
 - 권장값
 - `pf_err: 0.05` (5% 오차 허용)
 - `pf_z: 0.99` (99% 신뢰도)

- `recovery_alpha_slow` / `recovery_alpha_fast` — 납치 복구 속도
 - 'Kidnapped Robot' 문제: 로봇이 갑자기 엉뚱한 곳으로 옮겨지는 상황
 - 파티클의 평균 가중치 변화를 감시하여 위치 손실 여부 판단
 - 위치를 잃었다고 판단 → 지도 전체에 무작위 파티클 배포
 - 두 파라미터의 역할

파라미터	반응	역할
<code>recovery_alpha_fast</code>	단기 변화에 민감	빠르게 복구
<code>recovery_alpha_slow</code>	장기적 안정성 유지	서서히 복구

- `0.0` 설정 시 해당 기능 비활성화
- 권장값: `slow: 0.001`, `fast: 0.1`

Part 1-2. 모션 모델 파라미터

- 로봇이 움직일 때 발생하는 바퀴 미끄러짐과 오차(Odometry Noise)를 모델링
 - `robot_model_type` — 로봇 구동 방식 선택
 - `nav2_amcl::DifferentialMotionModel` → 차동 구동 (예: TurtleBot3)
 - `nav2_amcl::OmniMotionModel` → 전방향 구동
 - `alpha1` ~ `alpha4` — 오도메트리 노이즈 특성

파라미터	설명
<code>alpha1</code>	회전 시 발생하는 회전 오차 (제자리 회전 시 각도 오차)
<code>alpha2</code>	이동 시 발생하는 회전 오차 (직진 중 의도치 않은 회전)
<code>alpha3</code>	이동 시 발생하는 이동 오차 (인코더 오차로 거리 차이)
<code>alpha4</code>	회전 시 발생하는 이동 오차 (회전 중 중심 이탈)

- alpha 값 조정의 직관적 이해
 - 바닥이 미끄러운 환경 (예: 타일 바닥)
 - 바퀴가 헛돌 가능성 높음 → 오도메트리 신뢰도 하락
 - → `alpha` 값을 키워야 파티클이 더 넓게 퍼져 불확실성 감당 가능
 - 시뮬레이션 vs 실물 로봇

환경	alpha 설정	이유
Gazebo 시뮬레이션	낮게	오차 거의 없음
실물 로봇	높게	바퀴 슬립, 센서 노이즈 존재

- 권장 시작값: `alpha1` ~ `alpha4` 모두 `0.2`

Part 1-3. 센서 모델 파라미터

- LiDAR 센서 데이터를 지도와 어떻게 비교할지 설정
 - `laser_model_type` — 센서 모델 방식 선택
 - `likelihood_field` (권장)
 - 미리 계산된 지도 주변의 확률 필드 사용
 - 연산 속도 매우 빠름 → 실시간 주행에 적합
 - `beam`
 - 레이저 빔 하나하나를 물리적으로 시뮬레이션
 - 정확할 수 있지만 매우 느림 → 실시간 적용 어려움
 - 결론: 대부분의 경우 `likelihood_field` 사용

- 거리 범위 및 장애물 영향 범위 설정

- `laser_min_range` / `laser_max_range`
 - 로봇 몸체에 가려지는 **근거리** 또는 신뢰할 수 없는 **원거리** 제외
 - `-1.0` 설정 시 센서 스펙 전체 범위 사용
- `laser_likelihood_max_dist`
 - 지도 상의 장애물 주위로 얼마나 넓게 **확률 버퍼**를 돌지 결정
 - 값이 클수록 장애물 주변 매칭이 관대해짐
 - 권장값: `2.0` (m)
- 센서 스펙을 확인하고 노이즈가 심한 구간은 잘라내어 정확도 향상

- **z_hit / z_rand / z_short / z_max** — 빔 모델 혼합 비율
 - 레이저 빔 측정 결과를 4가지 원인으로 분해하는 비율

파라미터	의미	권장값
z_hit	장애물에 정확히 맞음 (가장 중요)	0.5
z_rand	알 수 없는 노이즈, 잘못된 반사광	0.5
z_short	사람 등 지도에 없는 물체가 빔을 차단	0.05
z_max	빔이 아무것도 못 찾고 최대 거리까지 도달	0.05

- 환경에 따른 조정
 - 유리창·움직이는 사람이 많은 환경 → z_rand 값 ↑
 - 깔끔한 실내 환경 → z_hit 값 ↑

Part 1-4. 업데이트 주기 파라미터

- `update_min_d` / `update_min_a` — 언제 파티클 계산을 수행할지 결정
 - `update_min_d` (이동 거리 기준)
 - 로봇이 설정된 거리(m)만큼 이동할 때만 업데이트
 - 권장값: `0.25` (25cm 이동 시마다 계산)
 - `update_min_a` (회전 각도 기준)
 - 로봇이 설정된 각도(rad)만큼 회전할 때만 업데이트
 - 권장값: `0.2` (약 11.5도 회전 시마다 계산)
- 주의사항

설정	문제
너무 작게 (자주 업데이트)	CPU 과부하 → 다른 노드 느려짐
너무 크게 (드물게 업데이트)	빠른 이동 시 위치 추적 실패

Part 2. TurtleBot3 권장 AMCL 설정 (YAML)

- 차동 구동(Differential) 방식 TurtleBot3에 최적화된 설정 예시

```
amcl:
  ros__parameters:
    # 1. 파티클 설정
    min_particles: 500          # 최소 파티클 수
    max_particles: 2000         # 최대 파티클 수
    pf_err: 0.05                # KLD 오차 기준
    pf_z: 0.99                  # KLD 확률 밀도
    recovery_alpha_slow: 0.001  # 느린 복구 속도 (0.0은 비활성)
    recovery_alpha_fast: 0.1    # 빠른 복구 속도 (0.0은 비활성)

    # 2. 모션 모델 설정 (차동 구동형)
    robot_model_type: "nav2_amcl::DifferentialMotionModel"
    alpha1: 0.2                 # 회전->회전 노이즈
    alpha2: 0.2                 # 이동->회전 노이즈
    alpha3: 0.2                 # 이동->이동 노이즈
    alpha4: 0.2                 # 회전->이동 노이즈
```

3. 센서 모델 설정

```
laser_model_type: "likelihood_field"
laser_min_range: -1.0           # 센서 스펙 최소값 사용
laser_max_range: -1.0           # 센서 스펙 최대값 사용
laser_likelihood_max_dist: 2.0  # 장애물 영향 범위
z_hit: 0.5                     # 실제 매칭 가중치
z_rand: 0.5                    # 무작위 노이즈 가중치
z_short: 0.05                  # 가려진 물체 가중치
z_max: 0.05                    # 최대 거리 가중치
```

4. 업데이트 주기

```
update_min_d: 0.25             # 25cm 이동 시마다 계산
update_min_a: 0.2              # 약 11.5도 회전 시마다 계산
```

5. 프레임 설정

```
base_frame_id: "base_footprint"
odom_frame_id: "odom"
global_frame_id: "map"
tf_broadcast: true              # map -> odom 변환 발행
```

Part 3. 글로벌 로컬라이제이션

글로벌 로컬라이제이션이란?

- 로봇이 자신의 위치를 전혀 모를 때 (Kidnapped Robot 상황) 위치를 처음부터 찾는 과정

- 서비스 호출로 시작

```
ros2 service call /reinitialize_global_localization std_srvs/srv/Empty {}
```

- 파티클 배포

- 서비스 호출 → 모든 파티클이 지도 전체 영역에 균등하게 무작위 배포

- 수렴 과정

- 로봇이 움직이며 LiDAR로 주변 스캔
- 지도 데이터와 비교 → 일치도 높은 파티클 가중치 ↑, 낮은 것 제거
- 반복 → 실제 위치로 파티클 구름이 수렴

- 올바른 글로벌 로컬라이제이션 수행 절차

1. 지도가 로딩된 상태에서 AMCL 노드를 실행
2. `/global_localization` 서비스를 호출하여 파티클을 지도 전체에 배포
3. 로봇을 제자리에서 회전(Spin) 시켜 주변 지형 스캔
4. 파티클이 특정 위치로 수렴하는지 확인

- 회전 전략의 효과

- LiDAR가 360도 방향의 고유한 지형 데이터를 빠르게 수집
- 파티클의 가중치 계산이 빠르게 이루어져 수렴 시간 단축

- RViz2 수렴 판단 기준

-  수렴 성공: 화살표(파티클)들이 로봇 주변으로 조밀하게 모여 작은 구름 형태
-  수렴 실패: 파티클이 지도 전체에 퍼져 있거나 여러 군데로 분산

- RViz에서 수동으로 초기 위치를 지정하는 절차

1. RViz 상단의 **2D Pose Estimate** 버튼 클릭
2. 지도상에서 실제 로봇의 위치와 **방향에 맞춰** 마우스 드래그
3. 로봇 주변에 파티클 클라우드가 형성되는 것 확인
4. 로봇을 조금씩 움직여 위치를 **정교화**

- 언제 사용하나?

- 로봇의 초기 위치를 대략 알고 있을 때 (글로벌 로컬라이제이션 대비 빠름)
- 자동 수렴이 느릴 때 수동으로 힌트를 주는 용도

Part 4. 로컬라이제이션 성능 평가

- 위치 추정의 정확도를 수치로 확인하는 방법

- `/amcl_pose` 토픽 — 공분산 행렬 모니터링
 - AMCL이 추정된 위치의 공분산 행렬을 발행
 - 대각 성분 값이 작을수록 위치 추정 정밀도가 높음

- `rqt_plot` — 실시간 수렴 시각화

```
rqt_plot /amcl_pose/pose/pose/covariance
```

- x축 위치 오차를 실시간 그래프로 확인
 - 그래프가 아래로 안정적으로 수렴하는지 확인이 핵심
- 판단 기준: 그래프가 낮은 값에서 안정적으로 유지 → 로컬라이제이션 성공

- 환경 변화로 인해 저장된 지도와 실제 환경이 달라지는 문제
 - AMCL의 내성 한계
 - 환경 변화 30% 이하: AMCL이 어느 정도 견딜 수 있음
 - 환경 변화 30% 초과: 파티클이 엉뚱한 곳으로 튀거나 로컬라이제이션 실패
 - 대응 전략
 - Beam Skip 설정 (`do_beamskip`)
 - 지도와 일치하지 않는 레이저 빔(새 장애물 등)을 계산에서 제외
 - Lifelong Mapping (SLAM Toolbox)
 - 로봇이 이동하며 변화된 환경을 실시간으로 지도에 반영
 - 동적 환경에서 장기 운용 시 권장



- 시뮬레이션과 실물 로봇의 파라미터 설정 차이

- 핵심 차이

항목	시뮬레이션 (Gazebo)	실물 로봇
오도메트리 오차	거의 없음	바퀴 슬립 발생
센서 노이즈	거의 없음	노이즈 존재

- 실물 로봇에서의 조정

- `alpha1` ~ `alpha4`: 시뮬레이션보다 높게 (바퀴 슬립 고려)
- `z_hit` 감소, `z_rand` 증가: 예기치 못한 센서 튜는 현상 대응
- `update_min_a`, `update_min_d` 조밀하게: 실시간성 향상